

Project Milestone 1 Submission

Microservices Architecture

Course: 420-N45-LA Web Services and Distributed Computing Winter 2026

Student Names: Noah Bouffard

Student Number: 2431848

Teacher: Professor Haikel Hichri

Submission Date: April 21, 2026

Quick Project Summary

This milestone decomposes the original auction lab monolith into a microservices architecture built with Spring Boot. The solution is organized around low-level services for auction orchestration, user management, listing management, and invoice and payment processing. An API gateway is used as the single external entry point, while each low-level microservice owns its own persistence layer and business rules.

The project is implemented as a multi-project Gradle build and is deployed with Docker Compose with one private database per backend service. The services are tested with repository-based integration tests and controller-based integration tests using JUnit Jupiter and WebTestClient. JaCoCo reports are generated for the documented low-level services to verify code coverage.

C4 Level 1 Explanation

The C4 Level 1 diagram presents the Auction Microservices System as a single software system from the user's point of view. Its purpose is to show the main actor, the platform user, and the high-level responsibility of the system without going into internal service details. At this level, the focus is on what the system does as a whole: it receives REST requests, applies auction-related business logic, and persists data through service-owned storage.

The Level 1 view also communicates that the platform is supported by separate databases rather than one shared monolithic schema. In this milestone, the backend services use service-owned databases to preserve bounded-context ownership and loose coupling. This diagram is useful because it gives a clear overview of the system boundary before the internal containers are introduced in the Level 2 diagram.

C4 Level 2 Explanation

The C4 Level 2 diagram decomposes the system into its main deployable containers. The API Gateway is the single public entry point and is responsible for routing requests to the correct low-level service while exposing a consistent external API. In this project, the low-level services are Auction Service, User Service, Listing Service, and Invoice Service, and each one is deployed independently behind the gateway.

The Auction Service manages auctions and bids. The User Service manages user account information and addresses. The Listing Service manages listing creation, updates, retrieval, deletion, and publishing workflow. The Invoice Service manages invoice creation, updates, retrieval, deletion, and payment workflow. Each of these services owns a private database,

which supports stronger bounded-context separation and makes the system easier to evolve toward a full microservices architecture.

This Level 2 design also reflects the milestone requirement that low-level services should not implement HATEOAS directly. Instead, low-level services return plain JSON responses and the gateway is the place where a more unified external API can be presented. Overall, the Level 2 view shows how the design diagrams were translated into independently deployable service containers.

DDD Domain Model Updates

The updated DDD domain model reorganizes the original monolithic auction application into separate bounded contexts that align with the milestone requirements. The Auction Orchestration bounded context owns the Auction aggregate, Bid entity, and auction lifecycle rules. The User Management bounded context owns the User aggregate and its related value objects such as Address, Email, and rating information. The Listing Management bounded context owns the Listing aggregate and the business rules around listing lifecycle and publishing.

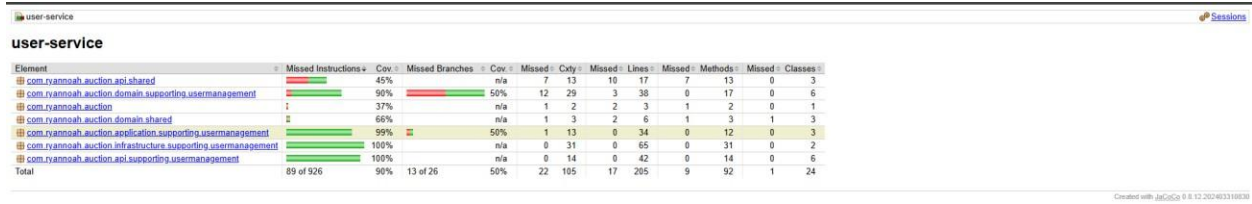
The Payment Processing bounded context owns the Invoice aggregate and the payment lifecycle, including the rules that determine when an invoice may be paid, updated, or deleted. Each bounded context also includes its own sub-domain specific exception, which improves the clarity of the domain model and provides service-specific error handling.

Examples in this project include AuctionHasBidsException, DuplicateEmailException, ListingAlreadyPublishedException, and InvoiceAlreadyPaidException.

Compared with the original lab monolith, the main design change is that responsibilities are now isolated inside separate bounded contexts with separate persistence. Milestone 1 does not require aggregates that depend on live data from another microservice, so the design avoids cross-service repository lookups. This makes the model cleaner for the first milestone and prepares the project for later extensions in Milestone 2.

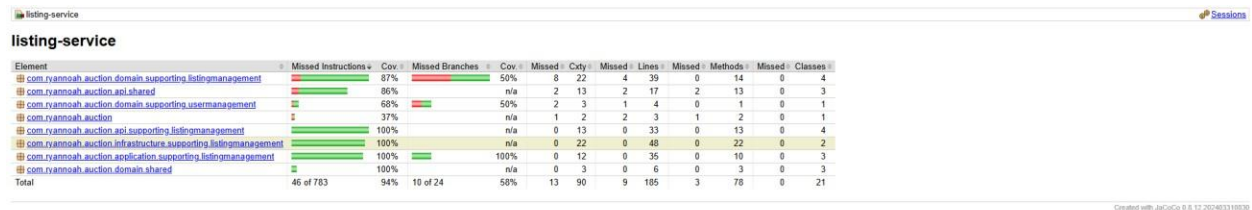
JaCoCo Screenshot Section

Screenshot 1 Title: User Service JaCoCo Report



Source: user-service/build/reports/jacoco/test/html/index.html

Screenshot 2 Title: Listing Service JaCoCo Report



Source: listing-service/build/reports/jacoco/test/html/index.html

Screenshot 3 Title: Invoice Service JaCoCo Report

Invoice-service

Sasports

invoice-service

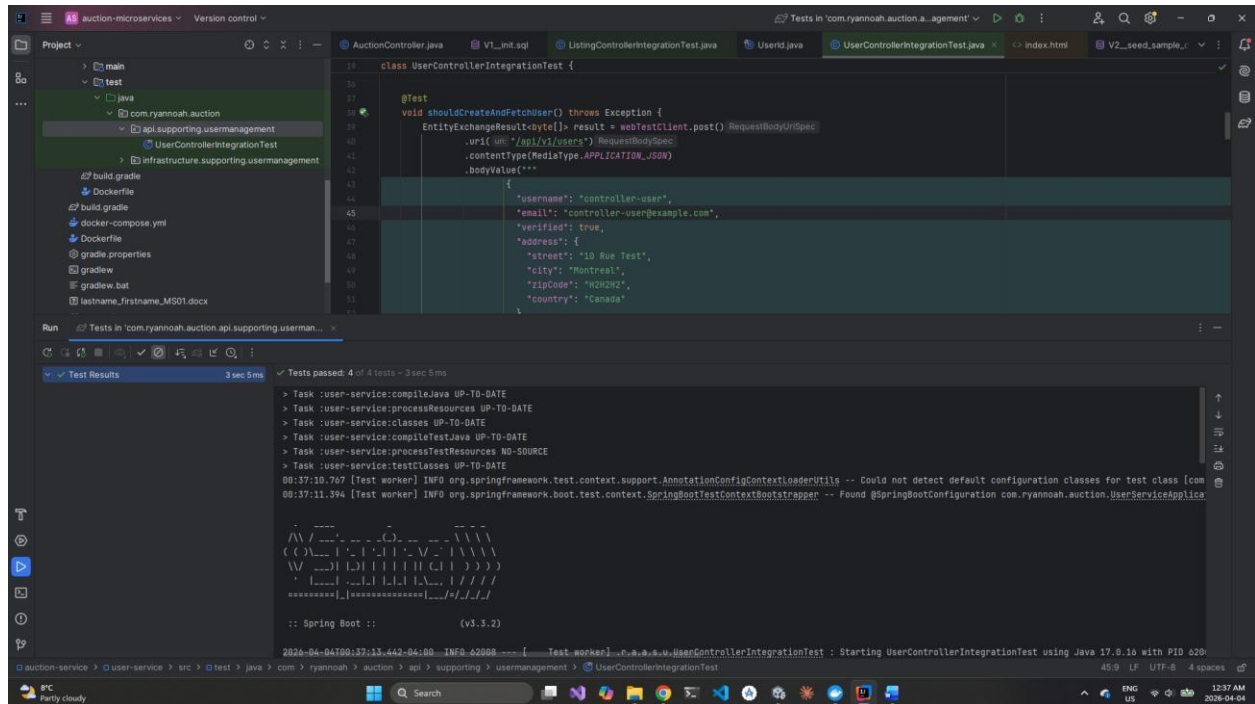
Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Cov	Missed	Lines	Missed	Methods	Missed	Classes	
com.ryanmoh.auction.domain.core.auctionorchestration		82%		61%	7	15	3	19	0	6	0	2	
com.ryanmoh.auction.application.supporting.paymentprocessing		97%		83%	1	13	1	42	0	10	0	3	
com.ryanmoh.auction.domain.supporting.paymentprocessing		97%		66%	2	21	1	48	0	18	0	6	
com.ryanmoh.auction.domain.supporting.usermanagement		68%		50%	2	3	1	4	0	1	0	1	
com.ryanmoh.auction		37%		n/a	1	2	2	3	1	2	0	1	
com.ryanmoh.auction.infrastructure.supporting.paymentprocessing		99%		59%	1	31	0	65	0	30	0	3	
com.ryanmoh.auction.acl.supporting.paymentprocessing		100%		n/a	0	11	0	35	0	11	0	4	
com.ryanmoh.auction.acl.shared		100%		n/a	0	13	0	17	0	13	0	3	
com.ryanmoh.auction.domain.shared		100%		n/a	0	3	0	6	0	3	0	3	
Total		37 of 1,628	96%	13 of 36	63%	14	112	8	239	1	94	0	26

Created with JacoCo 0.8.12.202403310830

Source: invoice-service/build/reports/jacoco/test/html/index.html

Domain-Specific Exception Test Screenshots

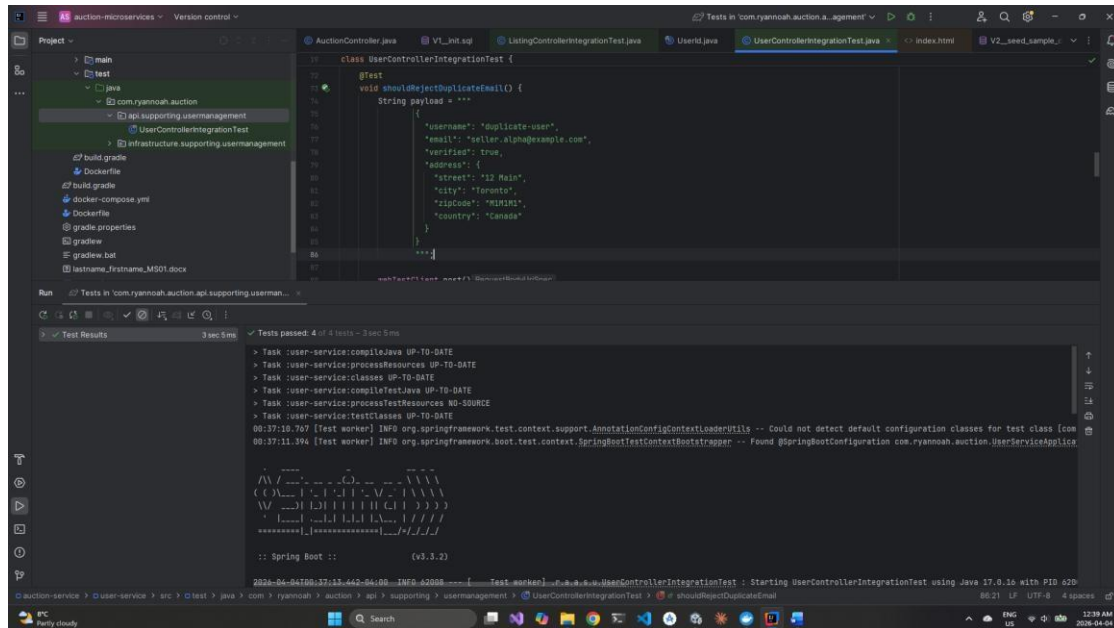
Screenshot 4 Title: User Service Positive Path - shouldCreateAndFetchUser



File: user -

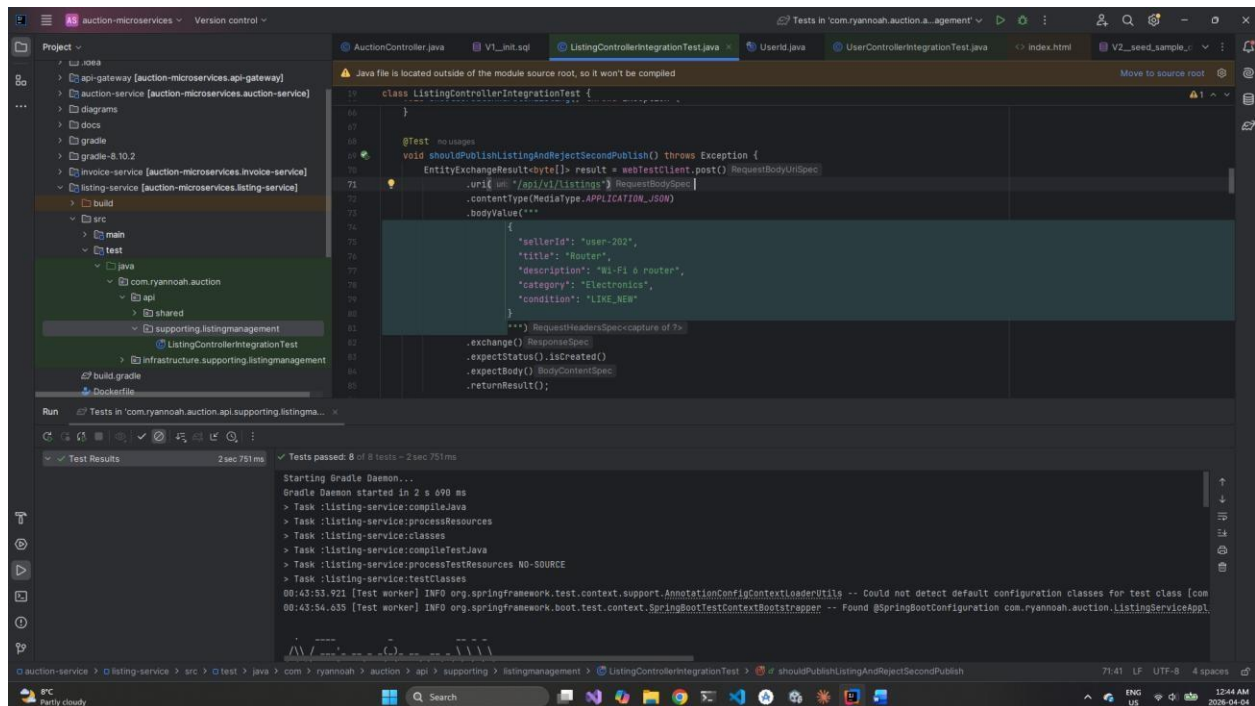
service/src/test/java/com/ryanhoah/auction/api/supporting/usermanagement/UserControllerIntegrationTest.java

Screenshot 5 Title: User Service Negative Path - shouldRejectDuplicateEmail



File: user-
service/src/test/java/com/ryanmoah/auction/api/supporting/usermanagement/UserContro
llerIntegrationTest.java

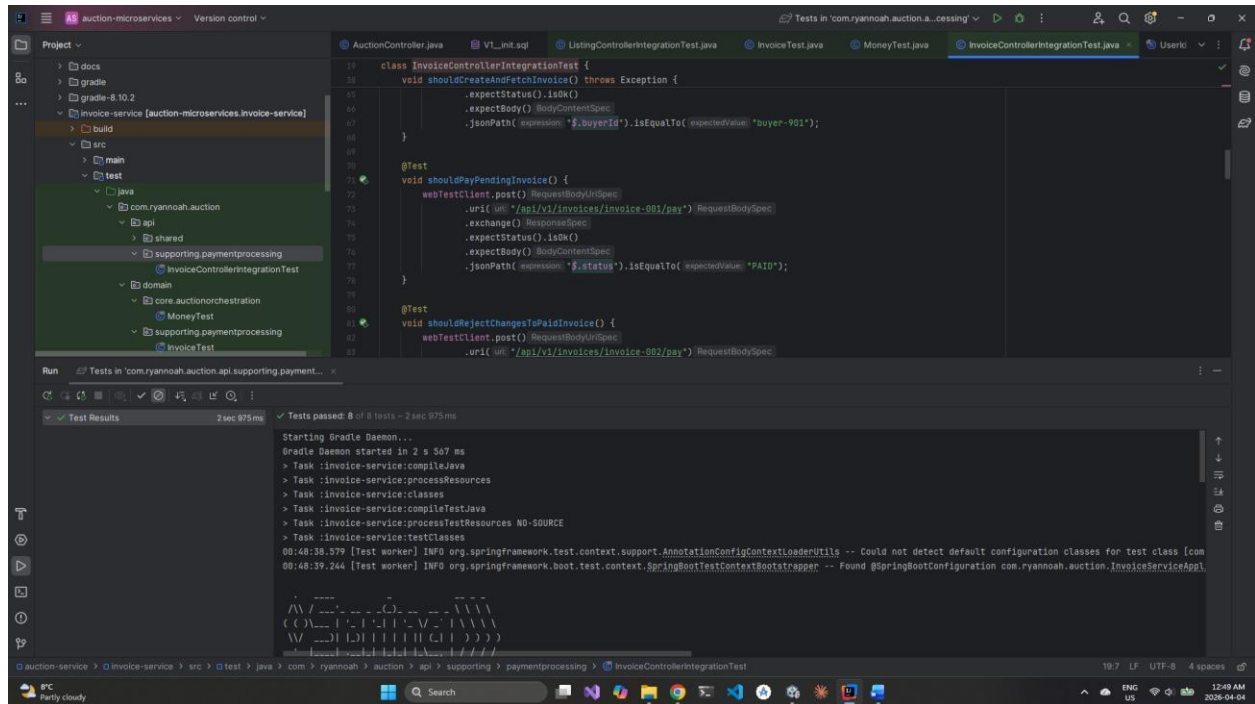
Screenshot 6 Title: Listing Service Positive and Negative Path - shouldPublishListingAndRejectSecondPublish



File: listing-

service/src/test/java/com/ryannoah/auction/api/supporting/listingmanagement/ListingControllerIntegrationTest.java

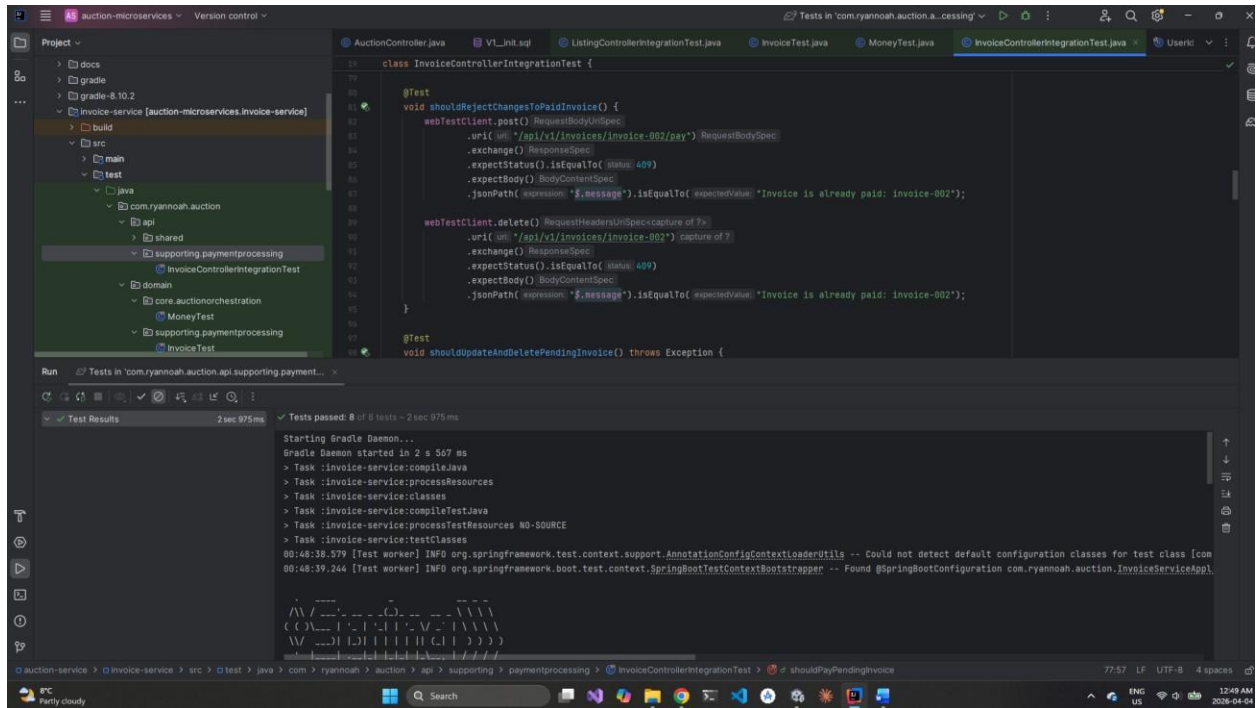
Screenshot 7 Title: Invoice Service Positive Path - shouldPayPendingInvoice



File: invoice-

service/src/test/java/com/ryannoah/auction/api/supporting/paymentprocessing/InvoiceControllerIntegrationTest.java

Screenshot 8 Title: Invoice Service Negative Path - shouldRejectChangesToPaidInvoice



File: invoice-

service/src/test/java/com/ryanhoah/auction/api/supporting/paymentprocessing/InvoiceControllerIntegrationTest.java